

Jev-with-the-Market — End-to-End Codebase Report

Jev-with-the-Market — End-to-End Codebase Report

Repo: github.com/manshu07/Jev-with-the-Market (a copy of JaiGanesh Kamalakannan's *Jev Meets the Market*; README links the original at [jai2010/Jev-Meets-the-Market](https://github.com/jai2010/Jev-Meets-the-Market))

Prepared for: Himanshu — covering what the code does, why every piece exists, verification I personally ran on the committed data, and exactly how to extend it to 5 years of data.

Verification basis: repo cloned and inspected line-by-line; offline re-simulation executed against the committed DuckDB decision database (no network, no Jev calls); full test suite executed (37/40 pass, 3 fail — explained in §8).

1. Executive summary

This repo is a **historical investment-decision experiment**, not a trading system. It asks: *can a probabilistic decision model — “Jev” — make a useful sequence of stock decisions when it must live with the consequences?* It gave Jev ₹10,00,000 and the NIFTY 100 universe, and let it decide BUY / HOLD / SELL / NO_ACTION at the end of each trading day, executing at the next session's open.

The four-layer pipeline:

- Phase 1 — Data:** download 18 months of daily OHLCV for the current NIFTY 100 from **Yahoo Finance** (yes — Yahoo Finance is the data source in this code, via the [yahoo-finance2](#) npm package) plus dividends/splits, and audit the data's quality.
- Phase 2 — Features:** turn raw prices into ~29 point-in-time indicators per stock per day (returns, SMAs, RSI, volatility, 52-week high distance, relative-to-index returns), stored in **DuckDB + Parquet**. Strict rule: a row is only “decision-ready” when every feature the prompt needs is computable *from data available that day*.
- Phase 3 — Jev smoke test:** send a 25-case sample (5 dates × 5 stock “archetypes”) to Jev through the **Vercel AI Gateway**, verify the answer schema, latency and repeat-consistency. No portfolio yet.
- Phase 4 — The experiment:** replay 118 real trading sessions (2026-03-10 → 2026-08-31). Each day, every decision-ready stock is sent to Jev as a structured **Choice question** (“which action?”) with the market + portfolio state. Answers, probabilities, raw payloads and latencies are persisted to DuckDB

with resume support; a frozen SHA-256 manifest locks the methodology before the first call. The result is published as JSON + markdown + a web replay UI.

Published result: Jev finished at **₹11,22,525 (+12.25%)** vs **₹10,11,109 (+1.11%)** for NIFTY 100 buy-and-hold, max drawdown -3.65%, across 10,299 Jev calls (0 failed), mean latency 545 ms, 19 executed trades.

What my verification found (details in §8): the Jev curve reproduces **exactly** from the committed decision database — the number is genuine to the stored decisions. But the published momentum/random baselines are flat at ₹10,00,000 while the committed code would have traded them to **+15.1% (momentum — beating Jev) and -9.4% (random)**; the frozen manifest's end date (09-22) doesn't match the published data (08-31) — over the full frozen window Jev lands at **+8.9%**, not +12.25%; and 3 shipped tests fail because of exactly this mismatch. The headline +12.25% is a partial-window result, and on the frozen full window the momentum baseline would have beaten it.

2. The stack and the core idea

Layer	Technology	Why this choice
Decision model	Jev (<code>typesafe-ai/jev</code>) via Vercel AI Gateway <code>POST /v1/evaluate</code>	Jev is TypeSafe's "System One" model: it doesn't generate text — it returns one typed, calibrated answer per question (a choice, a probability distribution over options, a confidence). That's exactly right for a decision loop: you get <code>BUY</code> + $P(\text{BUY})=0.57$ + probabilities for all four actions, in ~550 ms, deterministically structured — no JSON parsing of free-form LLM prose
App framework	Next.js 16 (App Router) + React 19 + TypeScript	The deliverable is a web experience: replay pages, portfolio view, trades, audit pages
Analytics store	DuckDB (<code>@duckdb/node-api</code>) + Parquet	Single-file embedded OLAP — 38,400 feature rows and 10,299 decision rows with zero infrastructure; also runs server-side in Next.js (<code>serverExternalPackages</code> in <code>next.config.ts</code>)
Market data	Yahoo Finance chart API via <code>yahoo-finance2</code>	Free, fast, has NSE (<code>*.NS</code>) symbols and the <code>^CNX100</code> index. Explicitly treated as <i>experimental</i> , not authoritative
Charts/UI	Recharts + Tailwind v4	Equity curves, sparklines, decision cards
Tests	Vitest (40 tests)	Indicators, Jev parsing, portfolio rules, replay engine, frozen-experiment consistency

Why a decision model and not ChatGPT? The README's own framing: the question was never "can Jev predict which stock goes up" but "can it make a *sequence* of decisions and own the outcomes". The design reflects that: every decision is recorded with its input state and probability, every execution is recorded with costs, and the whole run can be replayed and audited decision-by-decision.

What Jev (TypeSafe) actually returns here (`src/jev/schema.ts`, `src/jev/client.ts`): one request per (stock, day) containing:

- `state: { market, portfolio }` — the exact point-in-time features and portfolio context;
- `questions: { action: { type: "choice", instructions: DECISION_INSTRUCTIONS, criteria: { BUY: "...", HOLD: "...", SELL: "...", NO_ACTION: "..." } } }`.

The response contains `answers.action.choice` (one of the four actions) and `answers.action.probabilities` (a probability for **each** action, 0–1, validated client-side). The code stores both the *chosen-action probability* and the model's own raw `confidence` field separately, because they are different numbers — a deliberate honesty choice you can see in the reports.

The fixed instructions (`decision_schema_v1`, versioned — "do not edit this text for the same experiment id") tell Jev: use only the supplied state, assume nothing about the future, BUY ≠ "will rise", it means "taking a position is preferable given the evidence".

3. Architecture end-to-end

```
scripts/download-universe.mjs          (Phase 1 – npm run download)
  NSE ind_nifty100list.csv → config/universe.json (100 symbols)
  Yahoo chart API (.NS symbols, ^CNX100 benchmark, div|split events)
  → data/raw/ohlcv/*.csv (101 files) + data/raw/events/*.json +
trading_calendar.json
  → results/phase1_data_quality.{json,md}
  |
  ▼
scripts/features.ts → src/features/*    (Phase 2 – npm run features)
  read bars → align to benchmark calendar → computeStockFeatures()
  → 38,400 rows × 29 columns → validate → DuckDB market.duckdb + Parquet
  → results/phase2_summary.json (decision_ready_rows: 11,772)
  |
  ▼
scripts/jev-test.ts → src/jev/*          (Phase 3 – npm run jev:test)
  5 archetype stocks × 5 quantile dates → 25 cases (+5 repeats)
  → Vercel AI Gateway → store in jev_decisions.duckdb
  → results/phase3_jev_test.md (30 OK / 1 fail / latency 541 ms / 5/5 consistent)
  |
  ▼
scripts/phase4-run.ts → src/phase4/*    (Phase 4 – npm run phase4)
  frozen manifest (SHA-256 of universe+calendar+dataset+prompt+rules)
  → for each of 118 sessions: eligible stocks → Jev Choice call (resume-safe,
    concurrency 1, exponential backoff w/ Retry-After, account-block detection)
  → simulateAll(): three books (Jev / momentum / random) + benchmark
  decision today → execute next open (±5 bps slippage, 10 bps cost) → mark at
close
  → accounting assertions → publish experiment.json + report + replay data
  |
  ▼
Next.js app (app/experiments/*)
  /replay day-by-day animation of the recorded run (5-stage day loop)
  /portfolio /trades /analysis /phase3 static views over published JSON/DB
  /api/audit/day server route reading the DuckDB for per-day audit
```

Key structural decision: **the web app never calls Jev**. The experiment is run offline by scripts; the app only *replays the recording* (`data/processed/replay/experiment.json`, committed). That's why deployment is cheap and why results are reproducible.

4. Phase 1 — Data download, function by function

`scripts/download-universe.mjs` (699 lines). Design goal: *download once, cache hard, never re-fetch what's complete, and publish every data-quality finding instead of hiding it*.

Function	What it does	Why it exists
<code>readConstituents()</code>	Parses NSE's <code>ind_nifty100list.csv</code> , maps each to <code>{nse_symbol, yahoo_symbol (SYM.NS), name, industry, isin}</code> , hard-fails unless exactly 100 rows	Universe must be a fixed snapshot (2026-09-22), recorded in <code>config/universe.json</code> with an explicit <code>survivorship_note</code>
<code>fetchChart()</code>	Yahoo <code>chart()</code> call with <code>events: "div split"</code> , 4 attempts, quadratic backoff (750·attempt ² ms)	Yahoo intermittently 401/429s; retry must be polite but bounded
<code>writeSymbol()</code>	Writes CSV <code>date, open, high, low, close, adj_close, volume</code> deduped by session date (IST via <code>Intl.DateTimeFormat, en-CA</code> → YYYY-MM-DD), plus dividends/splits JSON	Raw cache is plain CSV — auditable in a text editor, diffable in git
<code>isComplete()</code>	Skips symbols whose cache reaches 2026-09-22 with >20 rows	Idempotent re-runs; <code>--force</code> overrides
<code>mapPool()</code>	Worker-pool with concurrency 5	101 symbols × ~1s without hammering Yahoo
<code>analyzeBars()</code>	Per symbol: null bars, OHLC violations (±0.05 tolerance), zero-volume days, negative prices, <code>adj_close≠close</code> days, dates missing vs the benchmark calendar	The quality gate that later justifies “close, not adj_close” and the placeholder-session filter
<code>splitBehavior()</code>	For each split event, compares the overnight close ratio to the split factor and classifies the series <code>split_adjusted</code> vs <code>unadjusted</code>	Yahoo NSE history turned out to be already split-adjusted — the report honestly documents the consequence: a split whose ex-date is <i>after</i> a simulated decision still distorts rupee price levels in the cached history

Function	What it does	Why it exists
<code>auditSessions()</code>	Walks all benchmark dates classifying each stock-day as real / flat-zero (OHLC = prev close, volume 0) / empty; detects placeholder sessions, later listings, post-listing holes	Found: 5 Yahoo placeholder dates dropped, 2026-01-01 index gap, 4 flat-print days (e.g. 96 symbols flat on 2025-03-18), 3 later listings (ENRIN, TATACAP, TMCV), and the known Vedanta 2026-04-30 bad print (−65% overnight, no split recorded) — the one data hole that survives into the experiment

The phase-1 report ends with the line *“No features were calculated. No portfolio was simulated. Jev was not called.”* — each phase states exactly what it did not do.

5. Phase 2 — Point-in-time features

`src/features/indicators.ts` + `build.ts` + `validate.ts`. Output: **38,400 rows (100 stocks × 384 sessions), 11,772 decision-ready, first ready date = 2026-03-10** — which is exactly why the experiment starts there (the warm-up window in `config/data.json` — history from 2025-03-03 — exists so that SMA-200 and 52-week-high are defined on the first decision day).

The feature set (all computed from `close`, never `adj_close`):

- **Returns:** 1d, 5d, 20d, 60d (`lagReturn`)
- **Trend:** SMA 20/50/200 and boolean `above_sma*` (`rollingMean` — returns null unless the *entire* window is present; one missing session poisons the window rather than silently shrinking it)
- **Momentum oscillator: Wilder RSI-14** (`rsiSeries`) — implemented with Wilder’s smoothing, and a subtle correctness decision: a missing close *ends the smoothing segment* instead of jumping the gap, because a data gap is not a knowable one-day return
- **Risk:** 20d realized volatility (sample stdev of daily returns), 20d drawdown from the 20d closing peak
- **Volume context:** 20d average volume + today/avg ratio
- **52-week structure:** rolling max of highs (252 sessions) and `distance_from_52w_high`

- **Market relative:** NIFTY 1d/5d/20d returns and the stock's 5d/20d return **minus** the index's (relative strength)

Bar validity (`isValidStockBar`) — this is the paper's spine. A bar enters any feature only if all five fields exist, are finite and positive, `high ≥ low`, and the bar isn't an inverted/impossible print. Flat zero-volume Yahoo prints (the 4 known dates) stay in the raw columns but are invisible to every window. The benchmark gets its own validator because indices legitimately have volume 0 — a high-low range is enough.

decision_ready — true only when all 12 prompt-critical features are non-null. This single boolean drives everything downstream: non-ready names are never sent to Jev, never eligible for the momentum baseline, and a held name with no decision that day is simply left unchanged.

Storage (`writeDatabase`) — DuckDB table `stock_features` via a typed appender, then `COPY ... TO parquet`. A post-write count check (`stored.rows must equal computed`) closes the loop against write corruption. `validate.ts` independently recomputes and diffs against the DB (`npm run validate` → PASS 38,400 rows).

6. Phase 3 — The Jev smoke test

`scripts/jev-test.ts` + `src/jev/{sample,state,store,hash}.ts`. Before risking a 118-day run, it answers three questions: does the gateway work, is the schema stable, and is the model *consistent* on identical input?

- **Sample construction** (`selectSample`): five dates at the 0/25/50/75/100% quantiles of sessions where ≥ 90 stocks are decision-ready; on the middle date pick one stock per archetype — *nearest 52w high, lowest RSI (oversold), strongest 60d uptrend, weakest 60d downtrend, flattest sideways* — then ask the same five tickers on all five dates = **25 cases**, plus 5 repeated inputs to measure consistency.
- **Mock portfolio** (`src/jev/state.ts`): a synthetic book (₹6L cash / ₹10L value / 2 of 5 positions) exists **only** so Jev sees realistic `currently_held` / `entry_price` / `holding_days` / `unrealized_return` context. First two tickers alphabetically are marked held; entry price is reconstructed as `close / (1 + return_20d)`. Clearly labelled “not the portfolio engine”.
- **Client** (`requestDecision`): 3 attempts, quadratic backoff, retry only on {408, 429, 500, 502, 503, 504}, special detection of `customer_verification_required` (account block → abort the run rather than burn the sample), strict response validation (unknown action or probability outside [0,1] = error, not a silent pass-through).
- **Every call is hashed** (`inputHash` = sha256 of market+portfolio+prompt version+model) and stored with full raw response, latency, attempt count — the audit trail that makes later phases replayable.

Result (committed): 30 OK / 1 fail, mean latency 541 ms, **5/5 repeat calls returned the identical action**, every action inside the schema.

7. Phase 4 – The experiment engine

`scripts/phase4-run.ts` orchestrating `src/phase4/{data,decisions,simulate,rules,publish,paths}.ts`.

7.1 The frozen manifest – the most important design decision

Before the first Jev call, `ensureManifest()` writes `data/processed/replay/manifest-v2.json`: experiment id `JEV-20260922-V2`, status FROZEN, start/end dates, all portfolio rules in prose, and **SHA-256 hashes** of the universe file, constituent CSV, trading calendar, feature DuckDB, decision prompt and rules. On every resume the hashes are recomputed – *any* methodology drift refuses to continue under the same run id. This is what separates an experiment from a vibe: you cannot accidentally change the prompt mid-run and keep the same result file.

7.2 The decision loop (`simulateAll`)

For each session *D* of 118, in order:

1. **Execute yesterday's plan** at today's opens (`apply()` → `executePlan()`): sells first, then buys; each fill gets slippage (BUY at `open × 1.0005`, SELL at `open × 0.9995`) and 10 bps transaction cost; shares are `floor(notional / cost-per-share)`; the resulting cash and positions are recorded as full trade records (decision date ≠ execution date is asserted).
2. **Mark to market** at today's closes (`markBook()` → `markToMarket()`); a missing close for a held position **throws** – the sim refuses to guess.
3. **Ask Jev** for every decision-ready stock (94–97 names typically): `marketState(features)` + real `portfolioPayload` (held? entry price? holding days? unrealized return? cash? position count?) → gateway call.
4. **Plan today's trades** (`planTrades()`): SELL signals on held names → sell list. BUY candidates ranked by **chosen-action probability desc, ticker asc** (a deterministic tie-break), capped to free slots (max 5 positions, buys on held names never add shares), each sized `min(20% × portfolio value, projected-cash-after-sells ÷ n)`.
5. The same machinery runs a **momentum baseline** (top-5 by 20d return, rebalanced daily) and a **random baseline** (seeded `mulberry32` PRNG reseeded per date – deterministic, reproducible), plus the benchmark scaled to ₹10L.

Decision-error policy is strict: an unresolved gateway call **stops the run** and is never silently recorded as NO_ACTION (`UnresolvedDecisionError`); resume reloads only `status='OK'` rows; DB writes are serialized through a promise-chain lock; progress logs every 30 calls with ETA. `accountingProblems()` then asserts every day: cash + market value = portfolio value (± 0.05), ≤ 5 positions, no negative cash, no zero/negative share counts, every trade's execution date strictly after its decision date. Only then does `publish()` write the replay JSON, the markdown report, and the equity curve.

7.3 The Jev retry layer (`src/phase4/decisions.ts`)

Hardened beyond Phase 3: 6 attempts, exponential backoff capped at 60 s, honouring HTTP `Retry-After` (seconds or date), concurrency-1 batches with per-ticker persistence, `UNIQUE(run_id, decision_date, ticker)` in the schema, and `assertNotInterruptedDatabase()` – which refuses to even open the abandoned V1 database by path. These are the scars of a real interrupted run: `data/processed/phase4/progress.json` shows the V1 run mid-flight at 2026-05-08 with 2,682 stored calls.

7.4 Publishing (`src/phase4/publish.ts`)

Converts day records into the replay dataset (positions valued, per-day decision list with probabilities, sparklines, highlighted best-confidence decision per action, executed trades) and renders the results report including: decision counts (BUY 4,284 / HOLD 529 / SELL 7 / NO_ACTION 5,479; mean chosen probability 0.701), forward returns after BUY/SELL at +1/+5/+20 sessions, and confidence-bucket vs subsequent-20-day-return tables – with the honest caption *“this is not a calibration test.”*

8. Verification I ran on the committed repo (evidence, not claims)

I re-ran the simulation **offline** – same `simulateAll()` code, Jev replaced by the **10,299 stored decisions** in the committed DuckDB – so the results below are driven purely by committed data plus committed logic:

Check	Result	Evidence
Jev curve reproduces	₹11,22,525 and 19 trades — exact match with the published run	<code>verify-offline.ts</code> output, window 2026-03-10→08-31; 13 stored-decision gaps mapped to the documented <code>NO_ACTION</code> policy
Momentum baseline	₹ Published flat ₹10,00,000 (+0.00%); the committed code trades it to ₹11,51,411 (+15.14%) — which beats Jev's +12.25%	Published <code>experiment.json</code> equity curve flat all run; my re-run moved from day 1; <code>scripts/audit-phase4-v2.ts</code> lines 94–95 stub <code>momentumEligible/randomEligible</code> to <code>[]</code> — the independent audit never audited the baselines
Random baseline	₹ Published flat ₹10,00,000; committed code takes it to ₹9,05,676 (–9.43%)	Same mechanism
Frozen window	₹ Manifest + <code>paths.ts</code> + 1 shipped test expect end 2026-09-22 (133 sessions); published data ends 2026-08-31 (118 sessions). Over the full frozen window my re-run lands Jev at ₹10,89,418 (+8.94%)	<code>npx vitest</code> → 3 failures, e.g. <code>expected '2026-08-31' to be '2026-09-22'</code>
Test suite	37/40 pass; the 3 failures are all this window/data mismatch, not logic bugs	<code>npm test</code>
Data-layer claims	₹ Verified against the raw cache: 101 CSVs (2025-03-03→2026-09-22), placeholder dates, Vedanta 2026-04-30 bad print, splits-already-adjusted finding	<code>results/phase1_data_quality.md</code> cross-checked against <code>data/raw/ohlcv/*.csv</code>

Reading of the evidence: the author ran the final V2 experiment, stopped it at 2026-08-31 (or the run took until then), published that partial window, and later hardened the code toward the full frozen window — the baseline integration and end-date belong to the post-freeze cleanup that the shipped data predates. The

decision log (10,299 stored calls with raw payloads) is genuine and internally consistent; the *comparison baselines* in the published UI are not meaningful, and the +12.25% headline is a partial-window number. On the frozen full window, momentum (+15.1%) would have beaten Jev — which, given the README's own “the possibility surprised me” framing, is exactly the kind of result the experiment was designed to be honest about.

9. How to add 5 years of data (Yahoo Finance — already the source in this code)

The good news: the architecture is *already built* for a longer window — the warm-up logic, feature windows and caching are all parameterised. The changes are concentrated in four files. Estimated effort: **half a day of code changes + a 15–20 hour machine-time run** (estimates below; treat as ranges, the Jev call count dominates).

Step 1 — widen the download window (`config/data.json`)

```
{
  "period1": "2021-09-23",           // 5y + 200-session warm-up before the
    experiment start
  "period2_exclusive": "2026-09-24",
  "window_note": "History starts ~14 months before the experiment so SMA-200 and
    the 52-week high are defined on day one."
}
```

Then `rm -rf data/raw/ohlcv data/raw/events && npm run download` (or let `isComplete()` skip nothing — the 2026-09-22 completeness check will fail and re-fetch). Concurrency 5 → roughly 101 symbols × ~1 s: **minutes, not hours**. Yahoo's chart API happily serves 5 years of daily bars per request; no pagination needed.

Step 2 — features and calendar (`scripts/download-universe.mjs`, `npm run features`)

The trading calendar and Phase-2 build read whatever the cache contains — no date edits needed. Data volume grows ~4× (≈1,230 sessions): DuckDB will hold ~130k feature rows without noticing. Re-run `npm run validate`. **Watch two things:** (a) flat zero-volume prints grow with history — the audit handles them, just re-read the new `phase1_data_quality.md`; (b) the split-adjustment finding compounds — more splits in 5 years, all pre-adjusted by Yahoo, same documented caveat.

Step 3 — the experiment window (`src/phase4/paths.ts`) and a new experiment id

```
export const START_DATE = "<your chosen start>" // ≥200 sessions after period1
export const END_DATE   = "2026-09-22"
// src/phase4/rules.ts: export const EXPERIMENT_ID = "JEV-<date>-V3"
```

New run id is non-negotiable: the manifest freezes hashes against the id, and the decisions table is keyed by `UNIQUE(run_id, decision_date, ticker)`. Delete/rename `data/processed/phase4/jev_experiment_v2.duckdb` (or point `PHASE4_DB` at a V3 file) so the old run's decisions don't leak into the new one via resume.

Step 4 — budget the Jev calls (this is the real cost)

Current run: 118 sessions × ~87 eligible stocks = 10,299 calls, mean 545 ms, serial (concurrency 1). Five years ≈ 1,225 experiment sessions → ≈ **107,000 calls**. Serial at 545 ms ≈ **16 hours**; the resume system means interruptions are free to restart. Raise `CONCURRENCY` in `phase4-run.ts` (it's a const, currently 1) to 4 → ~4–5 h, *if* the gateway's rate limits allow — the backoff machinery (Retry-After aware, cap 60 s) is already built for exactly this. Also decide the current-account-blocking question first: `customer_verification_required` aborts the run by design.

Also budget attention: ~107k decisions means the per-day decision tables in the published JSON grow ~10× — consider gzipping or summarising in the UI (the replay UI loads the whole `experiment.json`).

Broker data instead of Yahoo?

The README explicitly calls Yahoo “convenient for experimentation but not authoritative”. For NSE, the natural upgrades: **Zerodha Kite Connect** (₹2,000/mo, official historical candles API, requires a Zerodha account), **Upstox API** (free tier, historical V3), or **Fyers API** (free, historical candles). The integration point is small and clean: all three would replace **only** `scripts/download-universe.mjs`'s `fetchChart()` — if you keep the exact CSV contract (`date,open,high,low,close,adj_close,volume` + events JSON), **Phases 2–4 don't change at all**. That's the payoff of the phase separation. Broker data buys you: true unadjusted + adjusted series, corporate-action correctness (fixing the Vedanta-class bad prints), no Yahoo rate-limits. It does **not** buy you a fix for survivorship bias — that comes only from historical index-membership files (NSE publishes these; that's a separate, worthwhile upgrade).

Priority order if you do this

1. Fix the shipped inconsistencies first (they're cheap): publish the full frozen window or re-freeze the manifest to 08-31; make the audit script actually audit the baselines; get 40/40 tests green. Otherwise the 5-year run inherits a dishonest comparison.
2. Then extend to 5 years on Yahoo (fastest path, zero new credentials).
3. Swap in broker data when the experiment has proven it deserves the ₹2,000/mo.

10. Web app + deployment notes (brief, per your “leave it” on Vercel)

The app is five experiment pages over committed JSON/DuckDB plus one API route. Two deployment-relevant facts if you ever do host it: (1) `next.config.ts` `outputFileTracingIncludes` already pins every data file each route needs — including the `.duckdb.wal` — and DuckDB is marked `serverExternalPackages`; (2) pages that read data are `force-dynamic`. The one env var the *app* never needs is `AI_GATEWAY_API_KEY` — only the offline scripts do (server-only, in `.env.example`, with a “never expose to the browser” warning). The repo’s own deployment history (commits “fix: serve recorded replay without duckdb”, “Fix Phase 3 audit deployment”) shows the original author already fought exactly these Vercel packaging battles.

11. File map (where everything lives)

Path	Role
<code>scripts/download-universe.mjs</code>	Phase 1: Yahoo download + data-quality audit
<code>src/features/ {build, indicators, load, validate, report, constants}.ts</code>	Phase 2: feature computation + DuckDB write
<code>src/jev/{client, schema, state, sample, store, hash}.ts</code>	Jev gateway client, prompt, mock portfolio, sampling
<code>src/phase4/ {paths, data, decisions, rules, simulate, publish}.ts</code>	Experiment: window, data loading, retry layer, portfolio rules, sim loop, publishing
<code>scripts/{phase4-run, jev- test, features, validate, audit-phase4-v2}.ts</code>	Entry points (audit = read-only copy-then-check)
<code>data/raw/{ohlcv, events}/ , data/universe/</code>	101 cached price CSVs + events + NSE constituent file + trading calendar (all committed to git)
<code>data/processed/{market.duckdb, phase4/, replay/}</code>	Feature DB, decision DBs, published experiment JSON + frozen manifest
<code>results/phase{1,2,3,4}*. {md, json}</code>	Each phase's self-reported evidence
<code>app/experiments/ {replay, portfolio, trades, analysis, phase3}/</code>	Replay UI and static views
<code>tests/</code>	40 Vitest tests across indicators, Jev parsing, rules, replay, experiment consistency

12. Bottom line

- The codebase is a **well-engineered reproducibility shell** around a genuinely interesting question — frozen manifests, input hashing, resume-safe decision persistence, accounting assertions, per-phase “what this is not” honesty.
- **Yahoo Finance is confirmed as the market-data source** (`yahoo-finance2` → cached CSVs), used with unusual care (placeholder filtering, bar-validity gates, split-adjustment audit).
- The published **+12.25% is real to its stored decisions** (I reproduced it exactly) but is a **partial window** of the frozen experiment; on the full frozen window it's **+8.9%**, and the committed baseline code — which the published data flatlined — would have had **momentum at +15.1%**. Treat the README's comparison table as marketing until the repo's own 3 failing tests are made green.
- The 5-year extension you asked about is a **config-window change plus a new experiment id and ~107k Jev calls** (~16 h serial, ~4–5 h with concurrency), with an optional broker-data swap confined entirely to the Phase-1 downloader.

Not financial advice; this report analyses an experimental codebase and its committed artifacts only.